

조인 튜닝2

Simulation Test

모든 문제의 저작권은 아이티피이(ITPE)에 있습니다.

문제의 복제 및 제 3자 공유를 절대 금합니다.



1. 다음 SQL을 아래의 실행계획이 나오도록 유도할 때 (가)안에 넣을 적절한 힌트는?

[SQL]

```
SELECT EMPNO, ENAME
FROM EMP A
WHERE A. SAL > 3000
      AND EXISTS (SELECT /*+ (가) */ 1
                  FROM DEPT X
                  WHERE X.DEPTNO = A.DEPTNO)
```

[실행계획]

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		2	34	3 (0)	00:00:01
* 1	FILTER					
* 2	TABLE ACCESS FULL	EMP	7	119	3 (0)	00:00:01
* 3	INDEX UNIQUE SCAN	PK_DEPT	1	3	0 (0)	00:00:01

- ① no_unnest
- ② unnest
- ③ unnest push_subq
- ④ no_unnest push_subq

2. 다음 중 스칼라 서브쿼리의 특징을 설명한 것으로 가장 부적절한 것은?

- ① 스칼라 서브쿼리를 이용한 조인은 NL 조인처럼 한 레코드씩 순차적으로 진행한다.
- ② 메인 쿼리의 레코드마다 스칼라 서브쿼리를 통해 한 레코드의 하나의 값만 리턴한다.
- ③ 조인에 실패하면 NULL을 리턴한다.
- ④ 입력 값(=조인컬럼)과 출력 값(=리턴 값)을 Shared Pool에 있는 Result Cache에 캐싱한후 같은 입력 값에 대해서는 캐싱 된 값을 리턴하여 조인 부하를 줄인다.

3. 다음 SQL이 실행할 때 나타날 수 있는 실행계획으로 가장 적절한 것은?

```
SELECT EMPNO, ENAME
FROM EMP A
WHERE EXISTS (SELECT /*+ unnest nl_sj */1
              FROM DEPT X
              WHERE X.DEPTNO = A.DEPTNO AND X.DNAME LIKE 'A%')
```

① Execution Plan

```
-----
0      SELECT STATEMENT
1  0    NESTED LOOPS
2  1      TABLE ACCESS FULL OF EMP
3  1      TABLE ACCESS BY INDEX ROWID OF DEPT
4  2          INDEX UNIQUE SCAN OF PK_DEPT
```

② Execution Plan

```
-----
0      SELECT STATEMENT
1  0    NESTED LOOPS
2  1      TABLE ACCESS BY INDEX ROWID OF DEPT
3  2          INDEX UNIQUE SCAN OF PK_DEPT
4  1      TABLE ACCESS FULL OF EMP
```

③ Execution Plan

```
-----
0      SELECT STATEMENT
1  0    NESTED LOOPS SEMI
2  1      TABLE ACCESS FULL OF EMP
3  1      TABLE ACCESS BY INDEX ROWID OF DEPT
4  2          INDEX UNIQUE SCAN OF PK_DEPT
```

④ Execution Plan

```
-----
0      SELECT STATEMENT
1  0    NESTED LOOPS SEMI
2  1      TABLE ACCESS BY INDEX ROWID OF DEPT
3  2          INDEX UNIQUE SCAN OF PK_DEPT
4  1      TABLE ACCESS FULL OF EMP
```

4. 다음 SQL을 아래의 실행계획이 나오도록 유도할 때 (가)안에 넣을 적절한 힌트는?

[SQL]

```
SELECT A.EMPNO, A.ENAME, B.SAL
FROM EMP A
, (SELECT /*+ (가) */X.DEPTNO, SUM(X.SAL) AS SAL
    FROM EMP X, DEPT Y WHERE Y.DEPTNO = X.DEPTNO
    GROUP BY X.DEPTNO) B
WHERE A.SAL > 1000 AND B.DEPTNO = A.DEPTNO
```

[실행계획]

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		58	2088	7 (15)	00:00:01
1	HASH GROUP BY		58	2088	7 (15)	00:00:01
* 2	HASH JOIN		58	2088	6 (0)	00:00:01
* 3	TABLE ACCESS FULL	EMP	12	348	3 (0)	00:00:01
* 4	TABLE ACCESS FULL	EMP	14	98	3 (0)	00:00:01

- ① no_merge
- ② merge
- ③ no_unnest
- ④ unnest

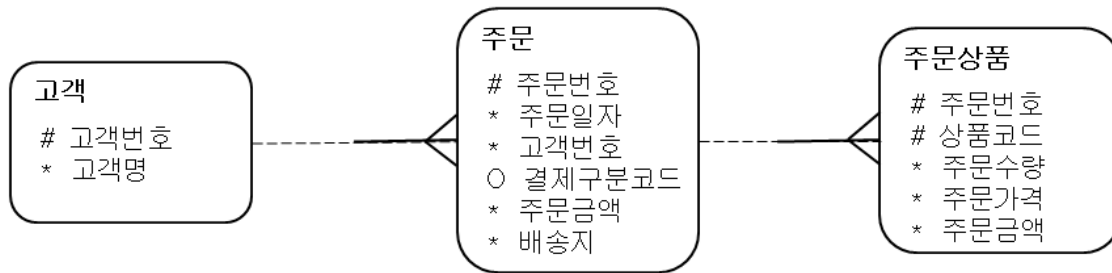
5. 다음 중 스칼라 서브쿼리의 특징을 설명한 것으로 가장 부적절한 것은?

```
SELECT EMPNO, ENAME
FROM EMP A
WHERE EXISTS (SELECT /*+ unnest nl_sj */1
              FROM DEPT X
              WHERE X.DEPTNO = A.DEPTNO AND X.DNAME LIKE 'A%')
```

- ① unnest 힌트는 옵티마이저가 서브쿼리를 조인 연산으로 변환하는 힌트이다.
- ② nl_sj 힌트는 Nested Loop 방식으로 조인을 하고 SEMI JOIN 방식으로 동작하는 힌트이다.
- ③ nl_sj 힌트를 nl_aj 힌트로 변경하면 성능상 더 유리하다.
- ④ 위 SQL은 쿼리변환이 실행된다.

6. 아래 조인문을 스칼라 서브쿼리로 변환할 때, 불필요한 요소가 전혀 없으면서 정확한 결과를 보장하는 SQL을 고르시오.

[ERD]



[SQL]

```

SELECT O.주문번호, MIN(O.고객번호) AS 고객번호, MIN(C.고객명) AS 고객명
      , MIN(O.주문금액) AS 총주문금액
      , MAX(P.주문금액) AS 최대주문금액
FROM 고객 C, 주문 O, 주문상품 P
WHERE O.주문일자 BETWEEN :orDt1 AND :orDt2
      AND C.고객번호 = O.고객번호
      AND P.주문번호 = O.주문번호
GROUP BY O.주문번호
    
```

- ① SELECT O.주문번호, O.고객번호
 , (SELECT 고객명 FROM 고객 WHERE 고객번호 = O.고객번호) AS 고객명
 , O.주문금액 AS 총주문금액
 , (SELECT 주문금액 FROM 주문상품 WHERE 주문번호 = O.주문번호) AS 최대주문금액
 FROM 주문 O
 WHERE 주문일자 BETWEEN :orDt1 AND :orDt2
- ② SELECT O.주문번호, O.고객번호
 , (SELECT 고객명 FROM 고객 WHERE 고객번호 = O.고객번호 AND ROWNUM <=1) AS 고객명
 , O.주문금액 AS 총주문금액
 , (SELECT 주문금액 FROM 주문상품 WHERE 주문번호 = O.주문번호 AND ROWNUM<=1) AS 최대주문금액
 FROM 주문 O
 WHERE 주문일자 BETWEEN :orDt1 AND :orDt2
- ③ SELECT O.주문번호, O.고객번호
 , (SELECT MAX(고객명) FROM 고객 WHERE 고객번호 = O.고객번호) AS 고객명
 , O.주문금액 AS 총주문금액
 , (SELECT MAX(주문금액) FROM 주문상품 WHERE 주문번호 = O.주문번호) AS 최대주문금액
 FROM 주문 O
 WHERE 주문일자 BETWEEN :orDt1 AND :orDt2

④ SELECT O.주문번호, O.고객번호
 , (SELECT 고객명 FROM 고객 WHERE 고객번호 = O.고객번호) AS 고객명
 , O.주문금액 AS 총주문금액
 , (SELECT MAX(주문금액) FROM 주문상품 WHERE 주문번호 = O.주문번호) AS 최대주문금액
 FROM 주문 O
 WHERE 주문일자 BETWEEN :orDt1 AND :orDt2

7. 다음 중 캐싱 효과 측면에서 스칼라 서브쿼리를 가장 적절히 사용한 ALIAS는?

[테이블 정보]

테이블명	데이터건수
결제구분	5
상품	100,000
고객	1,000,000
고객이력	10,000,000
주문	100,000,000
주문상세	500,000,000

[SQL]

SELECT O.주문번호, O. 주문일자, O.고객번호, O.주문금액, O. 배송지
 , (SELECT 고객명 FROM 고객 WHERE 고객번호 = O. 고객번호) 고객명
 , (SELECT 상품명 FROM 상품 WHERE 상품코드 = O. 상품코드) 상품명
 , (SELECT 결제구분명 FROM 결제구분
 WHERE 결제구분코드 = O. 결제구분코드) 결제구분명
 , (SELECT /*+ INDEX DESC(고객번호, 변경일시) 휴대폰번호
 FROM 고객이력 H
 WHERE 고객번호 = O. 고객번호 AND ROWNUM <= 1) 휴대폰번호
 FROM 주문 O, 주문상세 D
 WHERE O.주문일자 BETWEEN :orDt1 AND :orDt2
 AND D. 주문번호 = O. 주문번호

- ① 고객명 조회
- ② 상품명 조회
- ③ 결제구분명 조회
- ④ 휴대폰번호 조회

8. 다음 SQL 실행계획의 괄호 안에 들어갈 오퍼레이션을 가, 나, 다, 라, 마 순으로 올바르게 나열한 것을 고르시오.

[SQL]

```
SELECT C.고객번호, C.고객명
      , (SELECT ROUND(AVG(거래금액), 2) 평균거래금액
        FROM 거래
        WHERE 거래일시 >= TRUNC(SYSDATE, 'MM')
          AND 고객번호 = C. 고객번호)
FROM 고객 C
WHERE C.가입일시 >= TRUNC(ADD_MONTHS(SYSDATE, -1), 'MM')
```

[실행계획]

Execution Plan

0 SELECT STATEMENT Optimizer=ALL_ROWS

1 (가)

2 (나)

3 (다)

4 (라)

5 (마)

[오퍼레이션 항목]

A) INDEX (RANGE SCAN) OF '고객_IDX'

B) INDEX (RANGE SCAN) OF '거래_IDX'

C) TABLE ACCESS (BY INDEX ROWID) OF '고객'

D) TABLE ACCESS (BY INDEX ROWID) OF '거래'

E) SORT (AGGREGATE)

- ① C→A→E→D→B
- ② E→D→B→C→A
- ③ E→C→A→D→B
- ④ A→C→B→D→E

9. 다음 SQL에 대한 튜닝 방안으로 가장 부적절한 것을 고르시오.

[데이터]

지점 : 30개

판매내역 : 월 평균 3만 건

[인덱스 구성]

지점_PK : 지점코드

판매내역_PK : 판매지점코드 + 판매월

판매내역_X1 : 판매월

```
SELECT A.지점명, SUM(B.판매금액) 판매금액
FROM 지점 A, 판매내역 B
WHERE A.지점코드 = B.판매지점코드
AND B.판매월 BETWEEN '202101' AND '202103'
GROUP BY B.판매지점코드, A.지점명
ORDER BY 1, 2
```

Rows Row Source Operation

```
-----
      30    SORT GROUP BY
90000       NESTED LOOPS
90000       TABLE ACCESS BY INDEX ROWID 판매내역
90000       INDEX RANGE SCAN 판매내역_X1 (NONUNIQUE)
90000       TABLE ACCESS BY INDEX ROWID 지점
90000       INDEX UNIQUE SCAN 지점_PK (UNIQUE)
```

- ① GROUP BY의 'B.판매지점코드'를 'A.지점코드'로 변경한다.
- ② 조인 순서를 변경한다.
- ③ 해시 조인으로 유도한다.
- ④ 판매원 조건을 만족하는 판매내역을 지점코드로 GROUP BY 한 후 지정 테이블과 조인하도록 SQL을 변환한다.

10. 다음 SQL의 예상실행계획에서 나오는 오퍼레이션은?

```
SELECT A.EMPNO, A.ENAME, B.SAL1, B.SAL2
FROM EMP A
      , (SELECT /*+ no_merge push_pred */
          X.DEPTNO
          , AVG(X.SAL) AS SAL1
          , MAX(X.SAL) AS SAL2
        FROM EMP X
        GROUP BY X.DEPTNO) B
WHERE A.SAL > 1000
      AND A.DEPTNO = B.DEPTNO
```

- ① HASH GROUP BY
- ② VIEW
- ③ SORT ORDER BY
- ④ VIEW PUSHED PREDICATE

11. 다음 SQL의 예상실행계획에서 나오는 오퍼레이션에 HASH JOIN SEMI가 나왔다. (가)에 들어갈 힌트로 가장 적절한 것은?

```
SELECT EMPNO, ENAME
FROM EMP A
WHERE A. SAL > 3000
      AND EXISTS (SELECT /*+ (가) */1
                  FROM DEPT X
                  WHERE X.DEPTNO = A.DEPTNO AND X.DNAME LIKE 'A%');
```

- ① unnest hash_sj
- ② no_unnest hash_sj
- ③ unnest use_hash(X)
- ④ no_unnest use_hash(X)

12. 다음 SQL을 아래와 같이 튜닝 했을 때 개선 효과가 극대화되기 위해 만족해야 할 전제조건 중 부적절한 것은?

[튜닝 전 SQL]

```
SELECT C.고객번호, MIN(C.고객명) AS 고객명
      , SUM(O.주문금액) AS 주문금액
FROM 주문 O, 고객 C
WHERE O.주문일시 >= ADD_MONTHS(SYSDATE, -1)
      AND C.등록지점코드 = 'A10101'
      AND C.고객번호 = O.고객번호
GROUP BY O.고객번호
```

[튜닝 후 SQL]

```
SELECT C.고객번호, C.고객명, O.주문금액
FROM (
      SELECT 고객번호, SUM(주문금액) 주문금액
      FROM 주문
      WHERE 주문일시 >= ADD_MONTHS(SYSDATE, -1)
      GROUP BY 고객번호
    ) O
, 고객
WHERE C.고객번호 = O.고객번호
      AND C.등록지점코드 = 'A10101'
```

- ① 고객 당 하루 주문 건수가 많다.
- ② 주문일시 조건을 만족하는 데이터가 적다.
- ③ 등록지점코드 조건을 만족하는 데이터가 많다.
- ④ 주문 테이블이 고객 테이블보다 데이터가 많다.

13. 아래의 SQL을 튜닝하는데 가장 성능 향상 효율이 없는 것은?

[인덱스 구성]

고객_PK : 고객번호

거래_PK : 거래번호

거래_X1 : 거래일시

```
SELECT C. 고객번호, C. 고객명
      , (SELECT ROUND(AVG(거래금액), 2) AS 평균거래금액
        FROM 거래
        WHERE 거래일시 >= TRUNC(SYSDATE, 'MM')
        AND 고객번호 = C.고객번호) AS 평균거래금액
FROM 고객 C
WHERE C.가입일시 >= TRUNC(ADD_MONTHS(SYSDATE, -1), 'MM')
```

- ① 거래 테이블에 거래일시 + 고객번호 컬럼으로 인덱스를 생성한다.
- ② 고객 테이블에 가입일시 컬럼으로 인덱스를 생성한다.
- ③ 평균거래금액 서브쿼리가 캐싱효과가 크지 않으므로 조인으로 변경하는 것이 좋다.
- ④ unnest 힌트를 사용하면 부분범위처리가 가능하다.

14. 아래의 SQL에 대한 설명 중 적절한 것은?

[SQL]

```
SELECT A.구분코드
      , COUNT(B.신용카드ID) AS 신용카드건수
      , COUNT(C.계좌정보ID) AS 계좌정보건수
FROM 결제마스터 A
LEFT OUTER JOIN 신용카드 B ON A.결제정보ID=B.신용카드ID
LEFT OUTER JOIN 계좌정보 C ON A.결제정보ID=C.계좌정보ID
```

[테이블 구성]

- 결제마스터에는 구분코드값 1은 신용카드 결제정보, 구분코드값 2는 계좌번호 결제정보 저장

- ① OUTER JOIN을 INNER JOIN으로 변경하면 불필요한 SCAN을 줄일 수 있다.
- ② FULL 힌트를 사용하면 BLOCK 스캔범위를 줄일 수 있다.
- ③ OUTER JOIN 보다 결제마스터를 구분코드를 '1'과 '2'로 나누어 신용카드와 계좌번호 테이블을 조인하고 UNION ALL로 합치면 스캔범위를 줄일 수 있다.
- ④ USE_HASH 힌트를 사용하면 스캔범위를 줄일 수 있다.

15. 아래의 SQL을 튜닝하는데 가장 성능 향상 효율이 높은 것은?

[인덱스 구성]

고객_PK : 고객번호

고객_X1 : 가입일시

거래_PK : 거래번호

거래_X1 : 거래일시

거래_X2 : 고객번호 + 거래일시

```
SELECT C. 고객번호, C. 고객명, T. 평균거래, T. 최소거래, T. 최대거래
FROM 고객 C
      , (SELECT 고객번호
          , AVG(거래금액) AS 평균거래
          , MIN(거래금액) AS 최소거래
          , MAX(거래금액) AS 최대거래
        FROM 거래
        WHERE 거래일시 >= TRUNC(SYSDATE, 'MM')
        GROUP BY 고객번호) T
WHERE C. 가입일시 >= TRUNC(ADD MONTHS(SYSDATE, -1), 'MM')
      AND T.고객번호 = C.고객번호
```

- ① 서브쿼리에 no_merge, push_pred 힌트를 사용해 부분범위처리를 유도한다.
- ② 고객 테이블에 가입일시 + 고객번호 인덱스를 추가한다.
- ③ 거래 테이블에 거래일시 + 고객번호 인덱스를 추가한다.
- ④ 서브쿼리에 merge 힌트를 사용해 hash join을 유도한다.

16. 다음 힌트 설명 중 틀린것은?

- ① push_subq: 서브쿼리를 인라인 뷰로 푸시하여 메인 쿼리와 함께 최적화한다.
- ② unnest: 서브쿼리를 메인 쿼리로 풀어내어 조인으로 변환하는 과정을 말한다.
- ③ no_merge: 서브쿼리를 인라인 뷰로 병합하지 않고 그대로 유지하여 처리한다.
- ④ push_pred: 서브쿼리의 조건을 외부 쿼리로 푸시하여 전체 최적화를 유도한다.

17. 다음 중 스칼라 서브쿼리 캐싱에 대한 설명으로 틀린 것은?

- ① 스칼라 서브쿼리는 한 번 실행된 후 결과를 캐싱하여 동일한 입력에 대해 재사용될 수 있다.
- ② 스칼라 서브쿼리가 캐싱되면, 입력 데이터의 NDV가 클수록 캐싱으로 인한 성능 이점이 커진다.
- ③ 스칼라 서브쿼리 캐싱은 성능 최적화에 기여할 수 있다.
- ④ 동일한 스칼라 서브쿼리가 여러 번 호출될 경우 캐시된 결과를 사용하여 불필요한 재실행을 피할 수 있다.

18. 아래의 SQL을 튜닝하는데 가장 적절한 것은?

[SQL]

```
SELECT P.상품코드, MIN(P.상품명) 상품명, MIN(P.상품가격) AS 상품가격
      , SUM(O.주문수량) AS 총주문수량, SUM(O.주문금액) AS 총주문금액
FROM 주문상품 O, 상품 P
WHERE O.주문일시 >= ADD_MONTHS(SYSDATE, -1)
      AND O.할인유형코드 = 'K890'
      AND P.상품코드 = O.상품코드
GROUP BY P.상품코드
ORDER BY 총주문금액 DESC, 상품코드
```

Execution Plan

```
-----
0      SELECT STATEMENT Optimizer=ALL_ROWS (Cost=1M Card=20K Bytes=645K)
1 0      SORT (ORDER BY) (Cost=1M Card=20K Bytes=645K)
2 1      HASH (GROUP BY) (Cost=1M Card=20K Bytes=645K)
3 2      NESTED LOOPS (Cost=1M Card=497K Bytes=16M)
4 3      NESTED LOOPS (Cost=1M Card=500K Bytes=16M)
5 4      TABLE ACCESS (BY INDEX ROWID) OF '주문상품'(TABLE) (Cost=505K ...
6 5      INDEX (RANGE SCAN) OF '주문상품_X1' (INDEX) (Cost=3K Card=500K)
7 4      INDEX (UNIQUE SCAN) OF '상품_PK' (INDEX (UNIQUE)) (Cost=0 Card=1)
8 3      TABLE ACCESS (BY INDEX ROWID) OF '상품' (TABLE) (Cost=1 Card=1 ... )
```

[테이블 구성 및 데이터]

- 주문상품은 비파티션 테이블
- 한 달 주문상품 = 100만 건
- 주문상품의 보관 기간 = 10년
- 주문상품 총 건수 = 총 1억 2천만 건(= 100만 x 120개월)
- 할인유형코드 조건을 만족하는 데이터 비중 = 20%
- 등록된 상품 = 2만 개
- 2만 개 상품을 고르게 주문

[인덱스 구성]

상품_PK : 상품코드

주문상품_PK : 고객번호 + 상품코드 + 주문일시

주문상품_X1 : 주문일시 + 할인유형코드

- ① 주문상품 테이블을 서브쿼리로 재구성해 GROUP BY를 먼저 실행할 경우 조인건수는 줄어든다.
- ② 조인순서를 변경한다.
- ③ 주문상품 테이블에 상품코드 + 주문일시 + 할인유형코드 인덱스를 추가한다.
- ④ 주문상품 테이블을 FULL Scan하도록 FULL 힌트를 사용하는 것이 유리하다.

19. 아래 SQL을 튜닝하시오.

- 결과집합을 모두 출력하는 애플리케이션 환경을 기준으로 튜닝할 것
- 필요하다면, 인덱스 재구성 안을 제시할 것
- 튜닝 의도대로 정확히 실행되도록 옵티마이저 힌트를 기술할 것

[SQL]

```
SELECT P.상품코드, MIN(P.상품명) AS 상품명, MIN(P.상품가격) AS 상품가격
      , SUM(O.주문수량) AS 총주문수량, SUM(O.주문금액) AS 총주문금액
FROM 주문상품 O, 상품 P
WHERE O.주문일시 >= ADD_MONTHS(SYSDATE, -1)
      AND O.할인유형코드 = '0010'
      AND P.상품코드 = O.상품코드
GROUP BY P.상품코드
ORDER BY 총주문금액 DESC, P.상품코드
```

Execution Plan

```
-----
0      SELECT STATEMENT Optimizer=ALL_ROWS (Cost=279 Card=206 Bytes=11K)
1 0      SORT (ORDER BY) (Cost=279 Card=206 Bytes=11K)
2 1      HASH (GROUP BY) (Cost=279 Card=206 Bytes=11K)
3 2      NESTED LOOPS (Cost=277 Card=206 Bytes=11K)
4 3      NESTED LOOPS (Cost=277 Card=206 Bytes=11K)
5 4      PARTITION RANGE (ITERATOR) (Cost=70 Card=206 Bytes=9K)
6 5      TABLE ACCESS (BY LOCAL INDEX ROWID) OF '주문상품' (TABLE) ( ... )
7 6      INDEX (RANGE SCAN) OF '주문상품_X1' (INDEX) (Cost=34 Card=15)
8 4      INDEX (UNIQUE SCAN) OF '상품_PK' (INDEX (UNIQUE)) (Cost=0 Card=1)
9 3      TABLE ACCESS (BY INDEX ROWID) OF '상품' (TABLE) (Cost=1 Card=1 ... )
```

[테이블 구성 및 데이터]

- 주문상품은 월 단위 파티션 테이블(주문일시 기준)
- 한 달 주문상품 = 100만 건
- 주문상품의 보관 기간 = 10년
- 주문상품 총 건수 = 총 1억 2천만 건(= 100만 x 120개월)
- 할인유형코드 조건을 만족하는 데이터 비중 = 20%
- 등록된 상품 = 2만 개
- 대부분 상품을 한 달에 한 개 이상 주문

[인덱스 구성]

상품_PX : 상품코드

주문상품_PK : 고객번호 + 상품코드 + 주문일시

주문상품_X1 : 주문일시 + 할인유형코드

20. 아래 SQL을 튜닝하시오.

- 결과집합을 모두 출력하는 애플리케이션 환경을 기준으로 튜닝할 것
- 필요하다면, 인덱스 재구성 안을 제시할 것
- 튜닝 의도대로 정확히 실행되도록 옵티마이저 힌트를 기술할 것

[SQL]

```
SELECT P.상품코드, MIN(P.상품명) AS 상품명, MIN(P.상품가격) AS 상품가격
      , SUM(O.주문수량) AS 총주문수량, SUM(O.주문금액) AS 총주문금액
FROM 주문상품 O, 상품 P
WHERE O.주문일시 >= ADD_MONTHS(SYSDATE, -1)
      AND O.할인유형코드 = '0010'
      AND P.상품코드 = O.상품코드
GROUP BY P.상품코드
ORDER BY 총주문금액 DESC, P.상품코드
```

Execution Plan

```
-----
0      SELECT STATEMENT Optimizer=ALL_ROWS (Cost=204K Card=1 Bytes=34)
1 0      SORT (ORDER BY) (Cost=204K Card=1 Bytes=34)
2 1      HASH (GROUP BY) (Cost=204K Card=1 Bytes=34)
3 2      NESTED LOOPS (Cost=204K Card=100K Bytes=3M)
4 3          NESTED LOOPS (Cost=204K Card=100K Bytes=3M)
5 4              PARTITION RANGE (ITERATOR) (Cost=104K Card=100K Bytes=2M)
6 5                  TABLE ACCESS (BY LOCAL INDEX ROWID) OF '주문상품'(TABLE) ( ... )
7 6                      INDEX (RANGE SCAN) OF '주문상품_X1' (INDEX) (Cost=3k Card=109K)
8 4                          INDEX (UNIQUE SCAN) OF '상품_PK' (INDEX (UNIQUE)) (Cost=0 Card=1)
9 3                              TABLE ACCESS (BY INDEX ROWID) OF '상품' (TABLE) (Cost=1 Card=1 ...)
```

[테이블 구성 및 데이터]

- 주문상품은 월 단위 파티션 테이블(주문일시 기준)
- 한 달 주문상품 = 100만 건
- 주문상품의 보관 기간 = 10년
- 주문상품 총 건수 = 총 1억 2천만 건(= 100만 x 120개월)
- 할인유형코드 조건을 만족하는 데이터 비중 = 20%
- 등록된 상품 = 2만 개
- 할인유형코드 = '0010' 조건으로 판매되는 상품은 100여 개

[인덱스 구성]

상품_PK : 상품코드

주문상품_PX : 고객번호 + 상품코드 + 주문일시

주문상품_X1 : 주문일시 + 할인유형코드

21. 아래 SQL을 튜닝하시오.

- 결과집합을 일부(보통 상위 100개)만 출력하고 멈추는 애플리케이션 환경
- 필요하다면, 인덱스 재구성 안을 제시할 것
- 튜닝 의도대로 정확히 실행되도록 옵티마이저 힌트를 기술할 것

[SQL]

```
SELECT P.상품코드, MIN(P.상품명) AS 상품명, MIN(P.등록일시) AS 등록일시
      , MIN(P.상품가격) AS 상품가격, MIN(P.공급자ID) AS 공급자ID
      , SUM(O.주문수량) AS 총주문수량, SUM(O.주문금액) AS 총주문금액
FROM 주문상품 O, 상품 P
WHERE O.주문일시 >= ADD_MONTHS(SYSDATE, -1)
      AND O.할인유형코드 = '0010'
      AND P.상품코드 = O.상품코드
GROUP BY P.상품코드
ORDER BY 등록일시 DESC
```

Execution Plan

```
-----
0      SELECT STATEMENT Optimizer=ALL_ROWS (Cost=204K Card=1 Bytes=34)
1  0      SORT (ORDER BY) (Cost=204K Card=1 Bytes=34)
2  1      HASH (GROUP BY) (Cost=204K Card=1 Bytes=34)
3  2      NESTED LOOPS (Cost=204K Card=100K Bytes=3M)
4  3      NESTED LOOPS (Cost=204K Card=100K Bytes=3M)
5  4      PARTITION RANGE (ITERATOR) (Cost=104K Card=100K Bytes=2M)
6  5      TABLE ACCESS (BY LOCAL INDEX ROWID) OF '주문상품'(TABLE) ( ... )
7  6      INDEX (RANGE SCAN) OF '주문상품_X1' (INDEX) (Cost=3k Card=109K)
8  4      INDEX (UNIQUE SCAN) OF '상품_PK' (INDEX (UNIQUE)) (Cost=0 Card=1)
9  3      TABLE ACCESS (BY INDEX ROWID) OF '상품' (TABLE) (Cost=1 Card=1 ...)
```

[테이블 구성 및 데이터]

- 주문상품은 비파티션 테이블
- 한 달 주문상품 = 100만 건
- 주문상품의 보관 기간 = 10년
- 주문상품 총 건수 = 총 1억 2천만 건(= 100만 x 120개월)
- 할인유형코드 조건을 만족하는 데이터 비중 = 10%
- 등록된 상품 = 50만 개 / 속성 = 500개
- 대부분 상품을 한 달에 한 개 이상 주문

[인덱스 구성]

상품_PK : 상품코드

주문상품_PK : 고객번호 + 상품코드 + 주문일시

주문상품_X1 : 주문일시 + 할인유형코드